

# Logic Explanation

Approach, design decisions and validation for E3, M4 and H4

---

Round 1, Phase 2: Analytical Core

**Competition:** Data Vortex | AARUUSH'26

**Theme:** Rebuilding the Social Engine

**Team:** Entropy

**Members:** Saanvi Grover & Aditya Sharma

**Questions:** E3 · M4 · H4

**Database:** SQLite (Entropy\_social\_engine.db)

## Why These Three Questions

---

We chose one question per level that (a) demands a genuinely different SQL technique, (b) forces an explicit decision about the NULLs left by the corruption, and (c) can be checked against a finding from our Phase 1 EDA rather than only returning rows.

| Question                                    | Core technique                                         | Phase 1 finding it                        |
|---------------------------------------------|--------------------------------------------------------|-------------------------------------------|
| E3 Average engagement by platform           | GROUP BY + RANK() + cross-joined benchmark             | Platform choice do (Kruskal–Wallis $p =$  |
| M4 Platform behaviour, $\geq 30k$ followers | Join + filter + two aggregated CTEs joined on platform | Follower count has n man $\rho = 0.025$ ) |
| H4 Follower-to-engagement anomaly           | Three-level CTE chain + NTILE(10) + PERCENT_RANK()     | Engagement is inde “anomalies” should     |

## Shared Design Decisions

---

### Handling the corrupted NULLs

The cleaned dataset intentionally keeps NULLs in `platform` (1,784 posts), `likes` (1,814) and `text_content`. Two different rules are used, and the choice is deliberate:

- **Averages (E3, M4): exclude the row.** `AVG()` skips NULLs on its own, but `AVG(likes + shares + comments)` would silently drop a row from the total while still counting its shares in `AVG(shares)`. Filtering `likes IS NOT NULL` once in the `WHERE` clause keeps all four averages on the *same* denominator. Substituting 0 would bias every average downward by about 15%.
- **Sums per user (H4): COALESCE(likes, 0).** A user’s total is a sum across many posts. Dropping a post because its like count is corrupted would also discard its (valid) shares and comments, penalising users at random. Treating the missing likes as 0 keeps every post and the bias is uniform across users because the corruption is MCAR.

### Defining “engagement”

Everywhere, `engagement = likes + shares + comments`, as defined by question E2 in the brief. E3 also reports the three components separately because the question asks for them.

### Ranking inside SQL

Rankings are produced with `RANK() OVER (ORDER BY ...)` rather than relying on `ORDER BY` alone, so the rank survives if the consumer re-sorts the result and ties are made visible.

## E3 – Average Engagement by Platform

---

### Approach

1. **platform\_stats CTE:** filter to rows with a known platform and a non-NULL like count, then `GROUP BY platform` to compute post count and the four averages.
2. **overall CTE:** the grand mean total engagement over the same filtered rows. It is a single scalar, so a `CROSS JOIN` attaches it to every platform row.
3. **Final SELECT:** `RANK()` by average total engagement, plus `pct_vs_overall = percentage deviation from the grand mean`.

## Why add the benchmark column

Returning only the ranking answers “which platform is highest” but hides *by how much*. The extra column shows Instagram is first by just +0.78% and Twitter last at −1.42%, which is the real answer to the question.

## Validation

- Sum of `post_count` = 8,662 = 12,000 − 1,784 (NULL platform) − 1,814 (NULL likes) + 260 (both NULL), matching pandas.
- `avg_total_engagement` equals `avg_likes` + `avg_shares` + `avg_comments` for every row (linearity of the mean over the same denominator).
- Kruskal–Wallis across the five platforms on these rows:  $H = 2.97$ ,  $p = 0.56$ .

## M4 – Platform Behaviour by High-Follower Users

---

### Approach

1. **high\_follower\_posts CTE**: join `posts` to `users` on `user_id` (indexed on both sides), keep users with `follower_count`  $\geq 30000$ , and compute engagement per post. The same NULL rule as E3 applies.
2. **cohort\_by\_platform CTE**: aggregate per platform: distinct users, posts and average engagement.
3. **baseline\_by\_platform CTE**: the same per-platform average across *all* users.
4. **Final SELECT**: join cohort to baseline on `platform`, rank by the cohort average and report `cohort_minus_baseline`.

### Why compare against a baseline

“Which platform gives high-follower users the most engagement” only matters if high-follower users behave differently from everyone else. Without the baseline, Instagram at 4,145 looks like a finding; with it, the gap to the all-user Instagram average is +105 ( $\approx 2.6\%$ ), and three of the five platforms are *below* their baseline.

## Validation

- 594 users have  $\geq 30,000$  followers; the five `high_follower_users` counts (394–419) overlap because users post on several platforms.
- Kruskal–Wallis across platforms within the cohort:  $H = 6.22$ ,  $p = 0.18$ .
- Mann–Whitney U, cohort vs. all other users’ posts:  $p = 0.98$ ; the cohort is indistinguishable from the rest.

## H4 – Follower-to-Engagement Anomaly

---

### Approach: three levels of analysis

1. **Post  $\rightarrow$  user (user\_engagement)**: join and GROUP BY user to obtain `post_count` and `total_engagement`. Every one of the 1,500 users has at least one post, so an inner join loses nobody.
2. **User  $\rightarrow$  population (ranked\_users)**: `NTILE(10) OVER (ORDER BY total_engagement DESC)` assigns each user a decile; decile 1 is exactly the top 10% (150 users). `PERCENT_RANK()` gives

a finer percentile for reporting, and `COUNT(*) OVER ()` carries the population size without another subquery.

3. **Population** → **anomalies**: filter `engagement_decile = 1` AND `follower_count < 5000`, then `RANK()` the survivors and derive `avg_engagement_per_post`.

### Why NTILE rather than a hard-coded cut-off

A threshold such as `total_engagement > 43948` would be correct today and wrong the moment a row changes. `NTILE(10)` defines “top 10%” relative to the data, so the query remains valid on any refresh of the table. `PERCENT_RANK` is included alongside it because `NTILE` alone cannot show *how far* inside the top decile each user sits.

### Why report avg\_engagement\_per\_post

The question is phrased around *total* engagement, but  $\text{total} = \text{posts} \times \text{average}$ . Reporting both lets the reader see immediately whether a user is in the top decile because their posts perform unusually well or simply because they post more. All 16 anomalies average 2,939–4,849 per post, i.e. around the platform-wide mean of  $\sim 4,000$ , while posting 10–18 times against a population mean of 8.0.

### Validation

- Decile 1 contains exactly 150 users ( $1,500 / 10$ ).
- 131 users (8.73%) have  $< 5,000$  followers. Under independence the expected number in the top decile is  $150 \times 0.0873 = 13.1$ ; 16 are observed (binomial  $p = 0.39$ ).
- Spearman correlation, post count vs. total engagement:  $\rho = 0.89$ ; follower count vs. total engagement:  $\rho = -0.03$  ( $p = 0.30$ ).

## Tooling

---

SQLite 3 via Python `sqlite3` and `pandas` inside `Entropy_01_SQL_Analysis.ipynb`; statistical checks with `scipy.stats`; screenshots and charts rendered with `matplotlib` from the exact query result sets.